

PLAN DE COURS 1 : PROGRAMMATION ORIENTÉE OBJET (JAVA)

- **Poids horaire** : 60 heures (15 semaines × 4 heures)
- **Approche** : 40 % Théorie / 60 % Pratique (Développement d'une application d'entreprise)

DESCRIPTIF DU COURS

Ce cours introduit les concepts fondamentaux de la programmation orientée objet (POO) en utilisant le langage Java. L'étudiant apprendra à traduire une logique d'affaires en algorithmes robustes, à structurer son code à l'aide de classes et d'objets, et à appliquer les piliers de la POO (encapsulation, héritage, polymorphisme). À travers un projet pratique évolutif, les étudiants développeront une application console d'abord simple, puis entièrement modulaire et sécurisée contre les erreurs d'exécution.

STRUCTURE DES MODULES (JAVA)

Module 1 : Bases de la syntaxe et algorithmique en Java (12 heures)

- **Semaine 1 : Environnement et variables**
 - *Théorie* : Fonctionnement de la JVM (Java Virtual Machine), compilation vs interprétation. Types de données primitifs (int, double, boolean) vs objets (String).
 - *Laboratoire* : Installation du JDK et d'un IDE (ex. IntelliJ IDEA ou Eclipse). Écriture d'un premier programme, compilation et exécution en ligne de commande.
- **Semaine 2 : Structures de contrôle et conditions**
 - *Théorie* : Opérateurs logiques. Structures décisionnelles (if, else if, switch).
 - *Laboratoire* : Écriture d'algorithmes de validation d'entrées d'utilisateurs via la classe Scanner.
- **Semaine 3 : Structures répétitives (Boucles) et Tableaux**
 - *Théorie* : Boucles (for, while, do-while). Manipulation des tableaux à une dimension (Array).
 - *Laboratoire* : **Début du Projet Fil Conducteur (Système de gestion de stock d'une PME)**. Création d'une boucle principale pour afficher un menu interactif en console et stocker les premiers produits dans un tableau fixe.

Module 2 : L'Avènement de l'Objet — Classes et Encapsulation (12 heures)

- **Semaine 4 : Introduction aux classes et instances**
 - *Théorie* : Différence entre classe (moule) et objet (instance). Les attributs de classe et le rôle du constructeur (new).
 - *Laboratoire* : Modélisation de la classe Produit (id, nom, prix, quantité) avec constructeurs personnalisés.
- **Semaine 5 : Le pilier de l'Encapsulation**
 - *Théorie* : Niveaux de visibilité (public, private). Rôle et importance des accesseurs (getters) et mutateurs (setters).

- *Laboratoire* : Sécurisation de la classe Produit : interdire l'affectation d'un prix négatif ou d'une quantité sous zéro grâce aux mutateurs.
- **Semaine 6 : Gestion dynamique de données (ArrayList)**
 - *Théorie* : Limites des tableaux fixes vs flexibilité de la collection ArrayList. Introduction aux méthodes de recherche et de suppression.
 - *Laboratoire* : Migration du projet de gestion de stock : remplacement du tableau fixe par un ArrayList<Produit> pour permettre l'ajout illimité d'articles.

Module 3 : Polymorphisme et Mécanismes Avancés (16 heures)

- **Semaine 7 : Héritage et réutilisation de code**
 - *Théorie* : Concept de super-classe et sous-classe (mot-clé extends). Redéfinition de méthodes (@Override) et utilisation de super.
 - *Laboratoire* : Spécialisation des produits dans le projet : création des classes enfants ProduitPerissable (date d'expiration) et ProduitÉlectronique (garantie).
- **Semaine 8 : Examen Intra-trimestriel (4 heures)**
 - *Évaluation* : Examen pratique sur machine (Codage d'une structure de classes avec encapsulation complète).
- **Semaine 9 : Classes abstraites et Interfaces**
 - *Théorie* : Modélisation de concepts abstraits (mot-clé abstract). Rôle des interfaces comme contrats de comportement (implements).
 - *Laboratoire* : Création d'une interface Promotionnelle avec une méthode appliquerRabais(). Implantation de cette interface uniquement sur certaines sous-classes de produits.
- **Semaine 10 : Le Polymorphisme**
 - *Théorie* : Capacité d'un objet à prendre plusieurs formes. Liaison dynamique au moment de l'exécution.
 - *Laboratoire* : Parcours de l'inventaire global (ArrayList<Produit>) et déclenchement automatique du calcul des rabais spécifiques à chaque type de produit grâce au polymorphisme.

Module 4 : Robustesse et Gestion des Fichiers (20 heures)

- **Semaine 11 : Gestion des exceptions (Traitement des erreurs)**
 - *Théorie* : Compréhension des erreurs d'exécution. Bloc try-catch-finally. Les types d'exceptions (InputMismatchException, NullPointerException).
 - *Laboratoire* : Blindage de l'application : intercepter les entrées de lettres lorsque le système attend un prix (chiffre) pour éviter le plantage du programme.

- **Semaine 12 : Persistance des données (Entrées/Sorties)**

- *Théorie* : Introduction aux flux de données (*Streams*). Écriture et lecture de fichiers texte de base (FileWriter, BufferedReader).
- *Laboratoire* : Ajout d'une fonction "Sauvegarder" et "Charger" dans l'application pour écrire l'inventaire dans un fichier CSV local.

- **Semaine 13 : Introduction aux collections avancées et packages**

- *Théorie* : Différence entre Listes, Ensembles (Set) et Dictionnaires (Map). Organisation du code en *packages*.
- *Laboratoire* : Optimisation de l'application par la restructuration du code source en architecture modulaire (modèles, services, vue).

- **Semaine 14 : Revue de code, tests et optimisation**

- *Théorie* : Principes de refactorisation (*clean code*). Introduction rapide aux concepts de tests unitaires (JUnit).
- *Laboratoire* : Finalisation et empaquetage du projet d'entreprise sous forme de fichier .jar exécutable.

- **Semaine 15 : Évaluation finale de synthèse (4 heures)**

- *Évaluation* : Développement pratique autonome d'une application orientée objet à partir d'un cahier des charges d'entreprise complet.